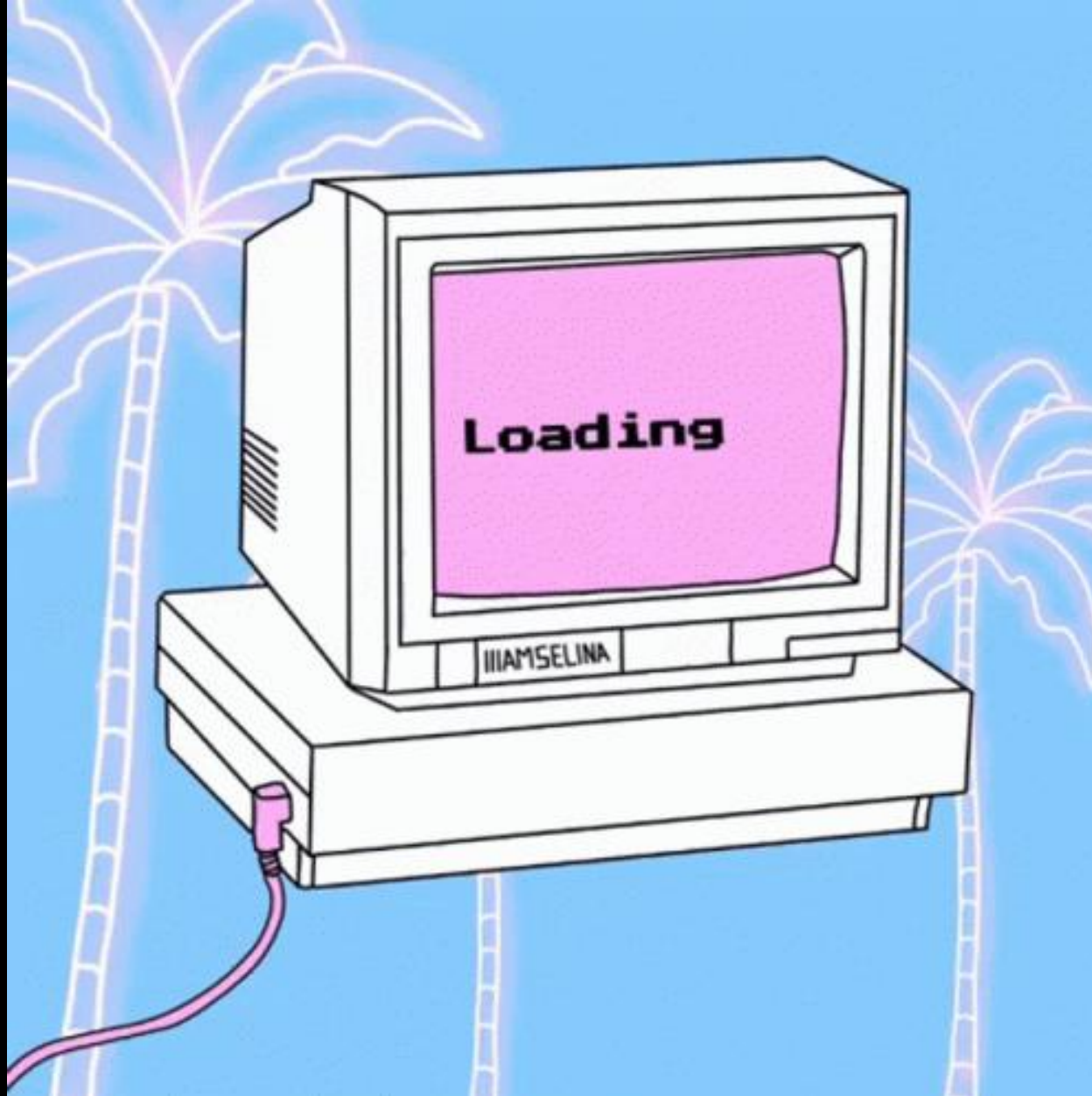




Universidad
Central



YA COMENZAMOS...



Universidad
Central

TÓPICOS AVANZADOS DE DATOS

2026-1



1. Información de contacto
2. Introducción a Paquetes en PL/SQL
3. Ejemplo Práctico: Creación y Uso de Paquetes
4. Excepciones Personalizadas en PL/SQL
5. Ejemplo Práctico: Definición y Manejo de Excepciones Personalizadas
6. Actividad Práctica: Desarrollo de Paquetes y Excepciones

Félix Nilo Bustios

Email: `felixjesusnilob@gmail.com`

Discord: Flex#5341

LinkedIn: Felix Nilo



1. Información de contacto
2. Introducción a Paquetes en PL/SQL
3. Ejemplo Práctico: Creación y Uso de Paquetes
4. Excepciones Personalizadas en PL/SQL
5. Ejemplo Práctico: Definición y Manejo de Excepciones Personalizadas
6. Actividad Práctica: Desarrollo de Paquetes y Excepciones



- Un paquete es una estructura que agrupa procedimientos, funciones, variables, cursores y excepciones relacionados en un solo objeto.
- Se compone de dos partes:
 - Especificación (PACKAGE): Define la interfaz pública (que se puede usar desde fuera).
 - Cuerpo (PACKAGE BODY): Contiene la implementación de los procedimientos y funciones declarados en la especificación.



- Ventajas de usar paquetes:
 - Modularidad: Organiza el código en unidades lógicas.
 - Encapsulamiento: Permite ocultar detalles de implementación (variables privadas).
 - Reutilización: Los procedimientos y funciones del paquete pueden ser usados por múltiples programas.
 - Mantenimiento: Facilita la actualización del código sin afectar a los usuarios del paquete.
 - Rendimiento: Las variables globales del paquete persisten en la sesión, reduciendo la necesidad de recalcular valores.



- Estructura básica de un paquete:

```
-- Especificación del paquete
CREATE OR REPLACE PACKAGE nombre_paquete AS
    PROCEDURE nombre_procedimiento(param1 IN tipo);
    FUNCTION nombre_funcion(param1 IN tipo) RETURN tipo;
END nombre_paquete;
/

-- Cuerpo del paquete
CREATE OR REPLACE PACKAGE BODY nombre_paquete AS
    PROCEDURE nombre_procedimiento(param1 IN tipo) IS
    BEGIN
        -- Implementación
        NULL;
    END nombre_procedimiento;

    FUNCTION nombre_funcion(param1 IN tipo) RETURN tipo IS
    BEGIN
        RETURN valor;
    END nombre_funcion;
END nombre_paquete;
/
```




- Ejemplo conceptual:
 - Un paquete `gestion_pedidos` podría incluir:
 - Procedimiento para registrar un pedido.
 - Función para calcular el total de un pedido.
 - Variable global para almacenar un contador de pedidos.



1. Información de contacto
2. Introducción a Paquetes en PL/SQL
3. Ejemplo Práctico: Creación y Uso de Paquetes
4. Excepciones Personalizadas en PL/SQL
5. Ejemplo Práctico: Definición y Manejo de Excepciones Personalizadas
6. Actividad Práctica: Desarrollo de Paquetes y Excepciones



EJEMPLO PRÁCTICO: CREACIÓN Y USO DE PAQUETES

- Escenario: Crear un paquete `gestion_pedidos` para manejar operaciones relacionadas con los pedidos en el esquema `curso_topicos`.
- Paso 1: Crear la especificación del paquete.

```
CREATE OR REPLACE PACKAGE gestion_pedidos AS
    -- Variable global
    g_contador_pedidos NUMBER := 0;

    -- Procedimiento para registrar un pedido
    PROCEDURE registrar_pedido(
        p_cliente_id IN NUMBER,
        p_total IN NUMBER,
        p_fecha IN DATE
    );

    -- Función para calcular el total de un pedido
    FUNCTION calcular_total_pedido(
        p_pedido_id IN NUMBER
    ) RETURN NUMBER;
END gestion_pedidos;
/
```



- Paso 2: Crear el cuerpo del paquete.

```
CREATE OR REPLACE PACKAGE BODY gestion_pedidos AS
    PROCEDURE registrar_pedido(
        p_cliente_id IN NUMBER,
        p_total IN NUMBER,
        p_fecha IN DATE
    ) IS
        v_pedido_id NUMBER;
    BEGIN
        -- Obtener el próximo PedidoID
        SELECT NVL(MAX(PedidoID), 0) + 1 INTO v_pedido_id
        FROM Pedidos;

        -- Insertar el pedido
        INSERT INTO Pedidos (PedidoID, ClienteID, Total,
FechaPedido)
        VALUES (v_pedido_id, p_cliente_id, p_total, p_fecha);

        -- Incrementar contador
        g_contador_pedidos := g_contador_pedidos + 1;

        DBMS_OUTPUT.PUT_LINE('Pedido ' || v_pedido_id || '
registrado. Total pedidos: ' || g_contador_pedidos);
    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error al registrar pedido: ' ||
SQLERRM);
```

```
        RAISE;
    END registrar_pedido;

    FUNCTION calcular_total_pedido(
        p_pedido_id IN NUMBER
    ) RETURN NUMBER IS
        v_total NUMBER := 0;
    BEGIN
        SELECT SUM(dp.Cantidad * p.Precio) INTO v_total
        FROM DetallesPedidos dp
        JOIN Productos p ON dp.ProductoID = p.ProductoID
        WHERE dp.PedidoID = p_pedido_id;

        RETURN NVL(v_total, 0);
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN 0;
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error al calcular total: ' ||
SQLERRM);
        RAISE;
    END calcular_total_pedido;
END gestion_pedidos;
/
```



- Paso 3: Probar el paquete.

```
-- Registrar un nuevo pedido
EXEC gestion_pedidos.registrar_pedido(1, 1500, TO_DATE('2025-06-09', 'YYYY-MM-DD'));

-- Calcular el total del Pedido 101
SET SERVEROUTPUT ON;
DECLARE
    v_total NUMBER;
BEGIN
    v_total := gestion_pedidos.calcular_total_pedido(101);
    DBMS_OUTPUT.PUT_LINE('Total del Pedido 101: ' || v_total);
END;
/
```



- Paso 3: Probar el paquete.

```
-- Registrar un nuevo pedido
EXEC gestion_pedidos.registrar_pedido(1, 1500, TO_DATE('2025-06-09', 'YYYY-MM-DD'));

-- Calcular el total del Pedido 101
SET SERVEROUTPUT ON;
DECLARE
    v_total NUMBER;
BEGIN
    v_total := gestion_pedidos.calcular_total_pedido(101);
    DBMS_OUTPUT.PUT_LINE('Total del Pedido 101: ' || v_total);
END;
/
```

- La variable `g_contador_pedidos` persiste durante la sesión, lo que permite mantener un conteo de pedidos registrados.
- Usa `DROP PACKAGE gestion_pedidos` para eliminar el paquete si es necesario.



1. Información de contacto
2. Introducción a Paquetes en PL/SQL
3. Ejemplo Práctico: Creación y Uso de Paquetes
4. Excepciones Personalizadas en PL/SQL
5. Ejemplo Práctico: Definición y Manejo de Excepciones Personalizadas
6. Actividad Práctica: Desarrollo de Paquetes y Excepciones



- Las excepciones personalizadas son errores definidos por el usuario para manejar situaciones específicas que no están cubiertas por las excepciones predefinidas de Oracle (como `NO_DATA_FOUND` o `DUP_VAL_ON_INDEX`).
- Se declaran con `EXCEPTION` y se lanzan con `RAISE`.



- Pasos para usar excepciones personalizadas:
 - Declarar la excepción en la sección DECLARE.
 - Asociarla a un código de error con PRAGMA EXCEPTION_INIT (opcional, para excepciones de Oracle).
 - Lanzarla con RAISE cuando se cumpla una condición específica.
 - Manejarla en la sección EXCEPTION.



- Ejemplo básico:

```
DECLARE
    e_pedido_invalido EXCEPTION;
    v_total NUMBER := -100;
BEGIN
    IF v_total < 0 THEN
        RAISE e_pedido_invalido;
    END IF;
EXCEPTION
    WHEN e_pedido_invalido THEN
        DBMS_OUTPUT.PUT_LINE('Error: El total del pedido no
puede ser negativo. ');
END;
/
```



- Asociar una excepción a un error de Oracle:

```
DECLARE
    e_clave_duplicada EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_clave_duplicada, -00001); -- ORA-
00001: unique constraint violated
BEGIN
    INSERT INTO Clientes (ClienteID, Nombre, Ciudad,
FechaNacimiento)
        VALUES (1, 'Cliente Duplicado', 'Santiago', TO_DATE('1990-
01-01', 'YYYY-MM-DD'));
EXCEPTION
    WHEN e_clave_duplicada THEN
        DBMS_OUTPUT.PUT_LINE('Error: ClienteID duplicado.');
```

END;
/



1. Información de contacto
2. Introducción a Paquetes en PL/SQL
3. Ejemplo Práctico: Creación y Uso de Paquetes
4. Excepciones Personalizadas en PL/SQL
5. Ejemplo Práctico: Definición y Manejo de Excepciones Personalizadas
6. Actividad Práctica: Desarrollo de Paquetes y Excepciones



EJEMPLO PRÁCTICO: DEFINICIÓN Y MANEJO DE EXCEPCIONES PERSONALIZADAS

- Escenario: Modificar el paquete `gestion_pedidos` para incluir excepciones personalizadas que validen:
 - El total del pedido no puede ser negativo.
 - El cliente debe existir antes de registrar un pedido.
- Paso 1: Actualizar la especificación del paquete.



EJEMPLO PRÁCTICO: DEFINICIÓN Y MANEJO DE EXCEPCIONES PERSONALIZADAS

```
CREATE OR REPLACE PACKAGE gestion_pedidos AS
    -- Excepciones personalizadas
    e_total_negativo EXCEPTION;
    e_cliente_no_existe EXCEPTION;

    -- Variable global
    g_contador_pedidos NUMBER := 0;

    -- Procedimiento para registrar un pedido
    PROCEDURE registrar_pedido(
        p_cliente_id IN NUMBER,
        p_total IN NUMBER,
        p_fecha IN DATE
    );

    -- Función para calcular el total de un pedido
    FUNCTION calcular_total_pedido(
        p_pedido_id IN NUMBER
    ) RETURN NUMBER;
END gestion_pedidos;
/
```



EJEMPLO PRÁCTICO: DEFINICIÓN Y MANEJO DE EXCEPCIONES PERSONALIZADAS

- Paso 2: Actualizar el cuerpo del paquete.

```
CREATE OR REPLACE PACKAGE BODY gestion_pedidos AS
    PROCEDURE registrar_pedido(
        p_cliente_id IN NUMBER,
        p_total IN NUMBER,
        p_fecha IN DATE
    ) IS
        v_pedido_id NUMBER;
        v_cliente_count NUMBER;
    BEGIN
        -- Validar total
        IF p_total < 0 THEN
            RAISE e_total_negativo;
        END IF;

        -- Validar existencia del cliente
        SELECT COUNT(*) INTO v_cliente_count
        FROM Clientes
        WHERE ClienteID = p_cliente_id;

        IF v_cliente_count = 0 THEN
            RAISE e_cliente_no_existe;
        END IF;

        -- Obtener el próximo PedidoID
        SELECT NVL(MAX(PedidoID), 0) + 1 INTO v_pedido_id
        FROM Pedidos;
```

```
-- Insertar el pedido
INSERT INTO Pedidos (PedidoID, ClienteID, Total, FechaPedido)
VALUES (v_pedido_id, p_cliente_id, p_total, p_fecha);

-- Incrementar contador
g_contador_pedidos := g_contador_pedidos + 1;

DBMS_OUTPUT.PUT_LINE('Pedido ' || v_pedido_id || ' registrado.
Total pedidos: ' || g_contador_pedidos);
EXCEPTION
    WHEN e_total_negativo THEN
        DBMS_OUTPUT.PUT_LINE('Error: El total del pedido no puede ser
negativo.');
```

```
        RAISE;
    WHEN e_cliente_no_existe THEN
        DBMS_OUTPUT.PUT_LINE('Error: El cliente no existe.');
```

```
        RAISE;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error al registrar pedido: ' || SQLERRM);
        RAISE;
    END registrar_pedido;

    FUNCTION calcular_total_pedido(
        p_pedido_id IN NUMBER
    ) RETURN NUMBER IS
        v_total NUMBER := 0;
    BEGIN
        SELECT SUM(dp.Cantidad * p.Precio) INTO v_total
        FROM DetallesPedidos dp
        JOIN Productos p ON dp.ProductoID = p.ProductoID
        WHERE dp.PedidoID = p_pedido_id;

        RETURN NVL(v_total, 0);
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN 0;
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error al calcular total: ' || SQLERRM);
            RAISE;
    END calcular_total_pedido;
END gestion_pedidos;
/
```



EJEMPLO PRÁCTICO: DEFINICIÓN Y MANEJO DE EXCEPCIONES PERSONALIZADAS

- Paso 3: Probar el paquete con excepciones.

-- Probar con un total negativo

```
EXEC gestion_pedidos.registrar_pedido(1, -500,  
TO_DATE('2025-06-09', 'YYYY-MM-DD'));
```

-- Probar con un cliente inexistente

```
EXEC gestion_pedidos.registrar_pedido(999, 1000,  
TO_DATE('2025-06-09', 'YYYY-MM-DD'));
```

-- Probar con datos válidos

```
EXEC gestion_pedidos.registrar_pedido(1, 1500,  
TO_DATE('2025-06-09', 'YYYY-MM-DD'));
```




1. Información de contacto
2. Introducción a Paquetes en PL/SQL
3. Ejemplo Práctico: Creación y Uso de Paquetes
4. Excepciones Personalizadas en PL/SQL
5. Ejemplo Práctico: Definición y Manejo de Excepciones Personalizadas
6. Actividad Práctica: Desarrollo de Paquetes y Excepciones



ACTIVIDAD PRÁCTICA: DESARROLLO DE PAQUETES Y EXCEPCIONES

1. Crea un paquete `gestion_clientes` con:
 - Un procedimiento `registrar_cliente` que reciba `ClienteID`, `Nombre`, `Ciudad` y `FechaNacimiento`, y valide que la fecha de nacimiento sea anterior a la fecha actual.
 - Una función `obtener_edad` que reciba un `ClienteID` y devuelva la edad del cliente.
 - Usa una variable global para contar los clientes registrados.



ACTIVIDAD PRÁCTICA: DESARROLLO DE PAQUETES Y EXCEPCIONES

```
-- Especificación
CREATE OR REPLACE PACKAGE gestion_clientes AS
    g_contador_clientes NUMBER := 0;
    PROCEDURE registrar_cliente(
        p_cliente_id IN NUMBER,
        p_nombre IN VARCHAR2,
        p_ciudad IN VARCHAR2,
        p_fecha_nacimiento IN DATE
    );
    FUNCTION obtener_edad(
        p_cliente_id IN NUMBER
    ) RETURN NUMBER;
END gestion_clientes;
/

-- Cuerpo
CREATE OR REPLACE PACKAGE BODY gestion_clientes AS
    PROCEDURE registrar_cliente(
        p_cliente_id IN NUMBER,
        p_nombre IN VARCHAR2,
        p_ciudad IN VARCHAR2,
        p_fecha_nacimiento IN DATE
    ) IS
    BEGIN
        IF p_fecha_nacimiento >= SYSDATE THEN
            RAISE_APPLICATION_ERROR(-20001, 'La fecha de
nacimiento debe ser anterior a la fecha actual.');
```

```
        g_contador_clientes := g_contador_clientes + 1;
        DBMS_OUTPUT.PUT_LINE('Cliente registrado. Total clientes: ' ||
g_contador_clientes);
    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error al registrar cliente: ' || SQLERRM);
            RAISE;
    END registrar_cliente;

    FUNCTION obtener_edad(
        p_cliente_id IN NUMBER
    ) RETURN NUMBER IS
        v_fecha_nacimiento DATE;
        v_edad NUMBER;
    BEGIN
        SELECT FechaNacimiento INTO v_fecha_nacimiento
        FROM Clientes
        WHERE ClienteID = p_cliente_id;

        v_edad := FLOOR(MONTHS_BETWEEN(SYSDATE, v_fecha_nacimiento) /
12);

        RETURN v_edad;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20002, 'Cliente no encontrado.');
```

```
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error al calcular edad: ' || SQLERRM);
            RAISE;
    END obtener_edad;
END gestion_clientes;
/
```

```
-- Prueba
EXEC gestion_clientes.registrar_cliente(4, 'Carlos Díaz', 'Santiago',
TO_DATE('1980-01-01', 'YYYY-MM-DD'));
DECLARE
    v_edad NUMBER;
BEGIN
    v_edad := gestion_clientes.obtener_edad(4);
    DBMS_OUTPUT.PUT_LINE('Edad del cliente 4: ' || v_edad);
END;
/
```



2. Modifica el paquete `gestion_clientes` para incluir una excepción personalizada `e_edad_invalida` que se lance si el cliente tiene menos de 18 años al registrarlo. Prueba el paquete con un cliente menor de edad.



ACTIVIDAD PRÁCTICA: DESARROLLO DE PAQUETES Y EXCEPCIONES

```
-- Especificación
CREATE OR REPLACE PACKAGE gestion_clientes AS
    e_edad_invalida EXCEPTION;
    g_contador_clientes NUMBER := 0;
    PROCEDURE registrar_cliente(
        p_cliente_id IN NUMBER,
        p_nombre IN VARCHAR2,
        p_ciudad IN VARCHAR2,
        p_fecha_nacimiento IN DATE
    );
    FUNCTION obtener_edad(
        p_cliente_id IN NUMBER
    ) RETURN NUMBER;
END gestion_clientes;
/

-- Cuerpo
CREATE OR REPLACE PACKAGE BODY gestion_clientes AS
    PROCEDURE registrar_cliente(
        p_cliente_id IN NUMBER,
        p_nombre IN VARCHAR2,
        p_ciudad IN VARCHAR2,
        p_fecha_nacimiento IN DATE
    ) IS
        v_edad NUMBER;
    BEGIN
        IF p_fecha_nacimiento >= SYSDATE THEN
            RAISE_APPLICATION_ERROR(-20001, 'La fecha de
nacimiento debe ser anterior a la fecha actual.');
```

```
        END IF;

        INSERT INTO Clientes (ClienteID, Nombre, Ciudad, FechaNacimiento)
        VALUES (p_cliente_id, p_nombre, p_ciudad, p_fecha_nacimiento);

        g_contador_clientes := g_contador_clientes + 1;
        DBMS_OUTPUT.PUT_LINE('Cliente registrado. Total clientes: ' ||
g_contador_clientes);
    EXCEPTION
        WHEN e_edad_invalida THEN
            años.');
```

```
        RAISE;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error al registrar cliente: ' || SQLERRM);
        RAISE;
    END registrar_cliente;

    FUNCTION obtener_edad(
        p_cliente_id IN NUMBER
    ) RETURN NUMBER IS
        v_fecha_nacimiento DATE;
        v_edad NUMBER;
    BEGIN
        SELECT FechaNacimiento INTO v_fecha_nacimiento
        FROM Clientes
        WHERE ClienteID = p_cliente_id;

        v_edad := FLOOR(MONTHS_BETWEEN(SYSDATE, v_fecha_nacimiento) /
12);

        RETURN v_edad;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20002, 'Cliente no encontrado.');
```